

INFO1167 Robotica

Laboratorio #1

Simulacion de Robots AGV en Bodega 2D

Documentacion completa del proyecto

Explicacion paso a paso del codigo y los algoritmos

Universidad Catolica de Temuco
Ingenieria Civil Informatica

1. Introduccion

Este documento explica paso a paso el proyecto desarrollado para el Laboratorio #1 del curso INFO1167 Robotica. El objetivo del laboratorio es simular el comportamiento de cuatro robots AGV (Automated Guided Vehicle) dentro de una bodega representada como una grilla bidimensional.

Un AGV es un vehiculo guiado automaticamente que se usa en centros de distribucion, bodegas y plantas de manufactura para transportar mercancia sin necesidad de un conductor humano. En este proyecto, los AGV deben:

- Iniciar en una base de carga ubicada en una esquina del mapa.
- Recibir asignacion aleatoria de un carrito que debe recoger.
- Calcular una ruta optima usando algoritmos de grafos.
- Moverse por la bodega rotando en pasos de 10 grados.
- Recoger el carrito (que cambia de color al del robot).
- Llevar el carrito a su destino y entregarlo.
- Volver a la base, recargar bateria al 100% y repetir el ciclo.

Este documento esta escrito para que cualquier persona, incluso sin conocimientos previos de programacion avanzada, pueda entender como funciona cada parte del codigo. Se usan analogias simples y se explica cada funcion linea por linea cuando es necesario.

2. Conceptos Previos

2.1. Grafos: la bodega como mapa de nodos

Para que un robot pueda moverse de un punto a otro de forma inteligente, primero necesitamos representar el espacio como un grafo. Un grafo es una estructura matematica formada por nodos (tambien llamados vertices) y aristas (conexiones entre nodos).

Imagina que la bodega es un tablero de ajedrez. Cada casilla es un nodo. Dos casillas adyacentes (arriba, abajo, izquierda, derecha) estan conectadas por una arista. Asi, moverse de la casilla (0,0) a la (1,0) es como caminar por una arista.

En este proyecto usamos la libreria NetworkX de Python para crear y manejar el grafo.

2.2. BFS - Busqueda en Anchura

BFS (Breadth-First Search) explora el grafo nivel por nivel, como cuando tiras una piedra al agua y las ondas se expanden en circulos. Primero visita todos los vecinos del nodo inicial, luego los vecinos de esos vecinos, y asi sucesivamente.

BFS encuentra la ruta con la menor cantidad de pasos (saltos), pero no considera la distancia real. Es util cuando todas las aristas tienen el mismo peso.

2.3. DFS - Busqueda en Profundidad

DFS (Depth-First Search) explora tan profundo como puede por un camino antes de retroceder. Es como caminar por un laberinto y seguir siempre la pared derecha hasta que llegas a un callejon sin salida; ahi retrocedes y pruebas otro camino.

DFS no garantiza la ruta mas corta, pero es util para explorar todas las posibilidades.

2.4. Dijkstra - La ruta mas corta con pesos

El algoritmo de Dijkstra encuentra la ruta de menor costo total desde un nodo de origen hasta todos los demas nodos. Usa una cola de prioridad (min-heap) para siempre expandir el camino mas prometedor primero.

En nuestro caso todas las aristas tienen peso 1.0 (cada paso cuesta lo mismo), por lo que Dijkstra y BFS dan el mismo resultado en cuanto a longitud. Sin embargo, Dijkstra es mas general porque funciona con pesos diferentes.

2.5. Rotacion y geometria

Los robots no se mueven instantaneamente: primero deben girar hacia la direccion deseada. Para saber hacia donde mirar, usamos la funcion `atan2(dy, dx)`, que devuelve el angulo en radianes entre el eje X positivo y el vector que apunta al siguiente nodo.

Como el laboratorio pide rotaciones en pasos de 10 grados, el robot gira 10 grados por cada tick de simulacion hasta que su angulo actual coincide (aproximadamente) con el angulo deseado. Solo entonces avanza al siguiente nodo.

3. Arquitectura del Proyecto

El proyecto esta organizado en modulos separados, cada uno con una responsabilidad unica:

src/agv_sim/modelos.py	-> Entidades de datos (Robot, Carrito, Config)
src/agv_sim/grafos.py	-> Algoritmos de grafos (BFS, DFS, Dijkstra)
src/agv_sim/simulacion.py	-> Motor de simulacion y maquina de estados
src/agv_sim/verificador.py	-> Verificacion de requisitos del laboratorio
src/agv_sim/visualizador.py	-> Exportar imagen estatica con matplotlib
src/agv_sim/animacion_vpython.py	-> Animacion 3D con Visual Python
src/agv_sim/animacion_tkinter.py	-> Animacion 2D con interfaz Tkinter
src/agv_sim/sprites.py	-> Generador de sprites con Pillow
demo.py	-> Demo completa de 5 pasos
tests/	-> Tests automaticos con pytest

El flujo de datos es el siguiente: la simulacion crea robots y carritos, usa los algoritmos de grafos para calcular rutas, avanza paso a paso (tick por tick), y los visualizadores leen el estado actual para dibujarlo en pantalla.

Separar el codigo en modulos tiene una ventaja enorme: puedes probar los grafos sin correr la simulacion, o probar la simulacion sin abrir ninguna ventana grafica. Esto es lo que hacen los tests automaticos.

4. Los Modelos de Datos

En programacion orientada a objetos, un 'modelo' es una clase o estructura que representa algo del mundo real. Aqui tenemos tres modelos principales: Robot, Carrito y ConfigSimulacion.

4.1. EstadoRobot (Enum)

Un Enum es una lista de valores constantes con nombre. En lugar de usar numeros magicos como 0, 1, 2, usamos nombres descriptivos que hacen el codigo mucho mas legible.

```
from enum import Enum

class EstadoRobot(str, Enum):
    EN_BASE = "en_base"
    BUSCANDO_CARRITO = "buscando_carrito"
    LLEVANDO_CARRITO = "llevando_carrito"
    VOLVIENDO_BASE = "volviendo_base"
    CARGANDO = "cargando"
```

Por ejemplo, cuando un robot esta cargando bateria, su estado es EstadoRobot.CARGANDO. Esto es mucho mas claro que si usaramos el numero 4.

4.2. Robot (Dataclass)

Un dataclass en Python es una forma concisa de crear clases que solo almacenan datos. Python genera automaticamente el constructor __init__, __repr__ y otros metodos utiles.

```
@dataclass
class Robot:
    id: int # Numero identificador (0, 1, 2, 3)
    color: str # Color asignado ("rojo", "azul", "verde", "naranja")
    base: tuple # Coordenadas (x, y) de la base de carga
    nodo: tuple # Coordenadas (x, y) actuales del robot
    angulo: float = 0.0 # Direccion en grados (0 = derecha, 90 = arriba)
    bateria: float = 100.0 # Porcentaje de bateria (0 a 100)
    estado: EstadoRobot = EstadoRobot.EN_BASE
    carrito_asignado: int | None = None # ID del carrito que lleva (None = ninguno)
    ruta: list = field(default_factory=list) # Lista de nodos por recorrer
    entregas_completadas: int = 0 # Contador de entregas exitosas
```

Cada robot empieza en su base con bateria al 100%, angulo 0 (mirando a la derecha) y sin ningun carrito asignado. A medida que avanza la simulacion, estos valores cambian.

4.3. Carrito (Dataclass)

```
@dataclass
class Carrito:
    id: int # Numero identificador
    color_original: str # Color actual (cambia al del robot asignado)
    nodo: tuple # Posicion actual en la grilla
    destino: tuple # Posicion donde debe ser entregado
    robot_asignado: int | None = None # ID del robot que lo transporta
    entregas: int = 0 # Veces que ha sido entregado
```

El campo `color_original` es clave: cuando un robot se asigna a un carrito, el carrito adopta el color del robot. Cuando se entrega, vuelve a ser gris. Esto permite ver visualmente cual robot lleva cual carrito.

4.4. ConfigSimulacion (Dataclass)

```
@dataclass
class ConfigSimulacion:
    ancho: int = 10          # Numero de columnas del mapa
    alto: int = 8            # Numero de filas del mapa
    cantidad_robots: int = 4 # Siempre 4 en este laboratorio
    cantidad_carritos: int = 4 # Numero de carritos en la bodega
    paso_rotacion: float = 10.0 # Grados que gira por tick
    gasto_bateria: float = 0.8 # Bateria consumida al moverse
    carga_bateria: float = 5.0 # Bateria recuperada por tick en base
    semilla: int = 42        # Semilla para numeros aleatorios (reproducibilidad)
    max_ticks: int = 1500    # Limite de ticks de la simulacion
    modo_ruta: str = "random_shortest" # Algoritmo de ruta
```

La semilla (semilla) garantiza que la simulacion sea determinista: si corres dos veces con `semilla=42`, obtendras exactamente el mismo resultado. Esto es fundamental para depurar y para la defensa, porque puedes repetir la demo con confianza.

5. Algoritmos de Grafos

Este modulo es el corazon matematico del proyecto. Aqui modelamos la bodega como un grafo y calculamos las rutas que los robots deben seguir.

5.1. Crear el grafo de la bodega

```
def crear_grafo_cuadrícula(ancho, alto):
    g = nx.Graph()
    # 1. Crear todos los nodos
    for x in range(ancho):
        for y in range(alto):
            g.add_node((x, y))
    # 2. Crear aristas horizontales y verticales
    for x in range(ancho):
        for y in range(alto):
            if x + 1 < ancho:
                g.add_edge((x, y), (x + 1, y), peso=1.0)
            if y + 1 < alto:
                g.add_edge((x, y), (x, y + 1), peso=1.0)
    return g
```

Este codigo crea un grafo donde cada nodo es una tupla (x, y). Dos nodos vecinos se conectan con una arista de peso 1.0. No se crean diagonales porque los robots solo se mueven horizontal o verticalmente.

Ejemplo visual para una grilla 3x3:

```
(0,2) -- (1,2) -- (2,2)
|       |       |
(0,1) -- (1,1) -- (2,1)
|       |       |
(0,0) -- (1,0) -- (2,0)
```

Cada linea es una arista por la que un robot puede moverse.

5.2. BFS - Búsqueda en Anchura

```
def bfs(grafo, origen, destino):
    from collections import deque
    if origen == destino:
        return [origen]
    q = deque([origen])          # Cola: primero en entrar, primero en salir
    anteriores = {}              # Guarda de donde venimos
    visitados = {origen}         # Para no visitar el mismo nodo dos veces
    while q:
        nodo = q.popleft()
        if nodo == destino:
            break
        for vecino in grafo.neighbors(nodo):
            if vecino in visitados:
                continue
            visitados.add(vecino)
            anteriores[vecino] = nodo
```

```
        q.append(vecino)
# Reconstruir el camino hacia atras
camino = [destino]
actual = destino
while actual != origen:
    actual = anteriores[actual]
    camino.append(actual)
camino.reverse()
return camino
```

La estructura deque es una cola doblemente terminada. Al usar popleft(), garantizamos que procesamos los nodos en el orden en que fueron descubiertos, lo que hace que BFS explore en anchura. El diccionario anteriores funciona como un mapa de 'de donde vine': para llegar al nodo (2,1) vine desde (1,1), etc.

5.3. DFS - Búsqueda en Profundidad

```
def dfs(grafo, origen, destino):
    pila = [(origen, [origen])]      # Cada elemento es (nodo_actual, camino_hasta_aqui)
    visitados = set()
    while pila:
        nodo, camino = pila.pop()    # pop() saca el ultimo = LIFO
        if nodo == destino:
            return camino
        if nodo in visitados:
            continue
        visitados.add(nodo)
        for vecino in grafo.neighbors(nodo):
            if vecino not in camino: # Evita ciclos inmediatos
                pila.append((vecino, camino + [vecino]))
    return []
```

Aqui usamos una pila (stack) en lugar de una cola. La diferencia clave es que pila.pop() saca el ultimo elemento agregado, por lo que siempre profundizamos por el ultimo vecino descubierto antes de probar otros caminos.

5.4. Dijkstra - Ruta mas corta

```
def dijkstra(grafo, origen, destino):
    if origen == destino:
        return [origen]
    cola = [(0.0, origen)]          # (distancia_acumulada, nodo)
    distancias = {origen: 0.0}      # Mejor distancia conocida hasta cada nodo
    anteriores = {}
    while cola:
        d, nodo = heapq.heappop(cola) # Saca el nodo con menor distancia
        if nodo == destino:
            break
        if d > distancias.get(nodo, float('inf')):
            continue                 # Ya encontramos una ruta mejor
        for vecino in grafo.neighbors(nodo):
            nd = d + grafo[nodo][vecino].get('peso', 1.0)
            if nd < distancias.get(vecino, float('inf')):
                distancias[vecino] = nd
                anteriores[vecino] = nodo
                heapq.heappush(cola, (nd, vecino))
    # Reconstruccion igual que en BFS
    camino = [destino]
    actual = destino
    while actual != origen:
        actual = anteriores[actual]
        camino.append(actual)
    camino.reverse()
    return camino
```

`heapq.heappop` siempre devuelve el elemento con la menor distancia. Esto garantiza que cuando llegamos al destino, ya tenemos la ruta de menor costo. La comprobacion `'if d > distancias.get(nodo, inf)'` es una optimizacion: si ya encontramos una ruta mejor a ese nodo, ignoramos esta entrada vieja de la cola.

5.5. Ruta corta aleatoria (random_shortest)

```
def ruta_corta_aleatoria(grafo, origen, destino, rng):
    if origen == destino:
        return [origen]
    # Distancia desde origen a todos los nodos
    dist_desde_origen = nx.single_source_dijkstra_path_length(grafo, origen, weight='peso')
    if destino not in dist_desde_origen:
        return []
    costo_total = dist_desde_origen[destino]
    # Distancia desde todos los nodos hasta destino
    dist_hasta_destino = nx.single_source_dijkstra_path_length(grafo, destino, weight='peso')

    camino = [origen]
    actual = origen
    while actual != destino:
        candidatos = []
        for vecino in grafo.neighbors(actual):
            costo_arista = grafo[actual][vecino].get('peso', 1.0)
            desde_origen = dist_desde_origen.get(actual, inf) + costo_arista
            hasta_destino = dist_hasta_destino.get(vecino, inf)
```

```
# Solo vecinos que mantienen la propiedad de ruta optima
if abs((desde_origen + hasta_destino) - costo_total) < 1e-9:
    candidatos.append(vecino)
if not candidatos:
    return []
actual = rng.choice(candidatos) # Elegimos aleatoriamente entre los optimos
camino.append(actual)
return camino
```

Este algoritmo es la estrella del proyecto. En lugar de devolver siempre la misma ruta optima, elige aleatoriamente entre todos los vecinos que mantienen la propiedad de 'ruta mas corta'. Matematicamente, un vecino es valido si:

$$\text{distancia}(\text{origen} \rightarrow \text{actual}) + \text{peso}(\text{arista}) + \text{distancia}(\text{vecino} \rightarrow \text{destino}) == \text{distancia_total}$$

Esto garantiza que el camino sigue siendo optimo, pero cada ejecucion puede tomar una ruta diferente. Asi los 4 robots no se amontonan en el mismo camino.

5.6. Enumerar todas las rutas

```
def todas_las_rutas(grafo, origen, destino, max_caminos=20, limite=6):
    caminos = []
    for i, camino in enumerate(
        nx.all_simple_paths(grafo, origen, destino, cutoff=limite)
    ):
        caminos.append(camino)
        if i + 1 >= max_caminos:
            break
    return caminos
```

NetworkX tiene una funcion `all_simple_paths` que enumera todos los caminos simples (sin ciclos) entre dos nodos. El parametro `cutoff=6` limita la longitud maxima para evitar explosion combinatorial. Esto demuestra que existen multiples rutas posibles entre dos puntos.

6. El Motor de Simulacion

La clase SimulacionAGV es el cerebro del proyecto. Controla todo: crea entidades, asigna misiones, mueve robots, consume bateria y registra eventos.

6.1. Inicializacion

```
class SimulacionAGV:
    COLORES = ["rojo", "azul", "verde", "naranja"]

    def __init__(self, config=None):
        self.config = config or ConfigSimulacion()
        self.rng = random.Random(self.config.semilla)
        self.grafo = crear_grafo_cuadricula(self.config.ancho, self.config.alto)
        self.robots = []
        self.carritos = []
        self.ticks = 0
        self.eventos = []
        self._iniciar_entidades()
```

`random.Random(semilla)` crea un generador de numeros aleatorios independiente. Usar una semilla fija hace que los numeros 'aleatorios' sean predecibles y reproducibles. El grafo se crea una sola vez al inicio porque la bodega no cambia.

6.2. Colocar robots y carritos

```
def _iniciar_entidades(self):
    esquinas = [
        (0, 0),
        (self.config.ancho - 1, 0),
        (0, self.config.alto - 1),
        (self.config.ancho - 1, self.config.alto - 1),
    ]
    bases = esquinas[: self.config.cantidad_robots]
    for i in range(self.config.cantidad_robots):
        base = bases[i % len(bases)]
        self.robots.append(
            Robot(id=i, color=self.COLORES[i], base=base, nodo=base)
        )
```

Los robots se colocan en las cuatro esquinas de la bodega. Cada uno tiene un color diferente y empieza en su base. Los carritos se colocan en posiciones aleatorias que no esten ocupadas por robots ni por otros carritos.

6.3. La magia: rotar y mover

```
def _rotar_y_mover(self, robot):
    if len(robot.ruta) <= 1:
        return False
    objetivo = robot.ruta[1] # El siguiente nodo en la ruta
    deseado = self._angulo_deseado(robot.nodo, objetivo)
```

```
delta = self._normalizar_angulo(deseado - robot.angulo)
paso = self.config.paso_rotacion # 10 grados
if abs(delta) > paso:
    # Todavía no mira al objetivo: girar 10 grados
    robot.angulo = self._normalizar_angulo(
        robot.angulo + (paso if delta > 0 else -paso)
    )
    return False # No avanzo todavía
# Ya mira al objetivo: avanzar
robot.angulo = deseado
robot.nodo = objetivo
robot.ruta = robot.ruta[1:] # Eliminar el nodo alcanzado
robot.bateria = max(0.0, robot.bateria - self.config.gasto_bateria)
return True # Si avanzo
```

Esta función es fundamental. Primero calcula el ángulo que debería tener el robot para mirar directamente al siguiente nodo (usando `atan2`). Luego calcula la diferencia entre el ángulo actual y el deseado. Si la diferencia es mayor a 10 grados, el robot solo gira 10 grados y NO se mueve. Cuando la diferencia es menor o igual a 10, el robot alinea su ángulo exactamente al deseado, avanza al siguiente nodo y consume batería.

Por que no se mueve inmediatamente? Porque el laboratorio pide explícitamente rotaciones en pasos de 10 grados. Esto simula un robot real que tiene que orientar sus ruedas antes de avanzar.

6.4. La maquina de estados del robot

Cada robot tiene un estado que dicta que debe hacer en cada tick. Los estados forman un ciclo cerrado: EN_BASE -> BUSCANDO_CARRITO -> LLEVANDO_CARRITO -> VOLVIENDO_BASE -> CARGANDO -> EN_BASE.

```
def _avanzar_robot(self, r):
    if r.estado == EstadoRobot.CARGANDO:
        r.bateria = min(100.0, r.bateria + self.config.carga_bateria)
        if r.bateria >= 100.0:
            r.estado = EstadoRobot.EN_BASE
            self.eventos.append(f"tick={self.ticks}: robot {r.id} bateria 100% listo")
        return

    if r.estado == EstadoRobot.EN_BASE:
        return # Esperando asignacion

    self._rotar_y_mover(r) # Intentar moverse

    if r.estado == EstadoRobot.BUSCANDO_CARRITO:
        carrito = self._carrito_por_id(r.carrito_asignado)
        if r.nodo == carrito.nodo:
            r.estado = EstadoRobot.LLEVANDO_CARRITO
            r.ruta = self._obtener_ruta(r.nodo, carrito.destino)
            self.eventos.append(f"tick={self.ticks}: robot {r.id} recogio carrito {carrito.id}")

    elif r.estado == EstadoRobot.LLEVANDO_CARRITO:
        carrito = self._carrito_por_id(r.carrito_asignado)
        carrito.nodo = r.nodo # El carrito se mueve con el robot
        if r.nodo == carrito.destino:
            # ENTREGA
            carrito.entregas += 1
            r.entregas_completadas += 1
            self.eventos.append(f"tick={self.ticks}: robot {r.id} entrego carrito {carrito.id}")
            # Resetear carrito
            carrito.nodo = self._nodo_aleatorio(...)
            carrito.destino = self._nodo_aleatorio(...)
            carrito.robot_asignado = None
            carrito.color_original = "gris"
            # Robot vuelve a base
            r.carrito_asignado = None
            r.estado = EstadoRobot.VOLVIENDO_BASE
            r.ruta = self._obtener_ruta(r.nodo, r.base)

    elif r.estado == EstadoRobot.VOLVIENDO_BASE:
        if r.nodo == r.base:
            r.estado = EstadoRobot.CARGANDO
            self.eventos.append(f"tick={self.ticks}: robot {r.id} llego a base y carga")
```

Cada estado tiene logica especifica. Cuando el robot llega al carrito, cambia de estado y recalcula la ruta hacia el destino. Cuando entrega, el carrito se reposiciona aleatoriamente y el robot inicia el retorno a base.

6.5. Asignacion de carritos

```
def _asignar_robots_libres(self):
    carritos_libres = [c for c in self.carritos if c.robot_asignado is None]
    self.rng.shuffle(carritos_libres)
    for r in self.robots:
        if r.estado not in {EstadoRobot.EN_BASE, EstadoRobot.CARGANDO}:
            continue
        if r.estado == EstadoRobot.CARGANDO and r.bateria < 100.0:
            continue
        if r.carrito_asignado is not None:
            continue
        if not carritos_libres:
            break
        carrito = carritos_libres.pop()
        carrito.robot_asignado = r.id
        carrito.color_original = r.color          # Cambia de color!
        r.carrito_asignado = carrito.id
        r.estado = EstadoRobot.BUSCANDO_CARRITO
        r.ruta = self._obtener_ruta(r.nodo, carrito.nodo)
        self.eventos.append(...)
```

Solo los robots en base o completamente cargados pueden recibir una mision. La lista de carritos libres se mezcla aleatoriamente para que la asignacion no sea siempre al mismo robot. El carrito cambia su color al del robot asignado.

7. Visualizacion

Una simulacion sin visualizacion es solo numeros. Este proyecto tiene tres formas de ver los resultados.

7.1. Imagen estatica con Matplotlib

```
def guardar_captura(sim, ruta_salida):
    fig, ax = plt.subplots(figsize=(10, 7))
    ax.set_title(f"Simulacion AGV (tick={sim.ticks})")
    for c in sim.carritos:
        ax.scatter(c.nodo[0], c.nodo[1], marker="s", s=120,
                   color=color(c.color_original), edgecolors="black")
        ax.scatter(c.destino[0], c.destino[1], marker="x", s=100, color="black")
    for r in sim.robots:
        ax.scatter(r.base[0], r.base[1], marker="^", s=160,
                   color=color(r.color), alpha=0.3)
        ax.scatter(r.nodo[0], r.nodo[1], marker="o", s=180,
                   color=color(r.color), edgecolors="black")
    fig.savefig(ruta_salida, dpi=180)
```

Matplotlib genera una imagen PNG con la disposicion final de todos los elementos. Es util para incluir en el informe o enviar por correo.

7.2. Animacion 3D con Visual Python

VPython abre una ventana del navegador con una escena 3D interactiva. Los robots se ven como cilindros de colores, los carritos como cajas, y los destinos como anillos negros. El usuario puede rotar la camara con el mouse.

7.3. Animacion 2D con Tkinter y Sprites propios

Como VPython a veces falla en Windows, creamos una alternativa completa usando Tkinter, la libreria de interfaces graficas incluida en Python. Pero en lugar de usar circulos y cuadrados abstractos, generamos sprites propios con Pillow.

Sprites con Pillow

```
def generar_robot(tam=64, color="rojo", angulo=0):
    img = Image.new("RGBA", (tam, tam), (0, 0, 0, 0))
    draw = ImageDraw.Draw(img)
    cx = tam * 0.5
    cy = tam * 0.58 # Centrado mas abajo para espacio arriba
    # ... dibujar ruedas, cuerpo, cabeza, antena, ojos rotados, boca
    return img
```

Cada sprite es una imagen PNG generada programaticamente. El robot tiene ruedas, cuerpo, cabeza redonda, antena con luz, ojos que miran hacia la direccion del movimiento, y una sonrisa. Las posiciones son proporcionales al tamaño, así que nunca se corta ninguna parte.

Los sprites se generan en memoria y se convierten a PhotoImage de Tkinter. No necesitamos archivos .png externos. Esto hace que el proyecto sea autocontenido: solo necesitas el código fuente y las dependencias de Python.

8. Verificacion y Tests

El archivo verificador.py revisa que todos los requisitos del laboratorio se cumplan. Hay 8 requisitos (R1 a R8) y cada uno se verifica con datos reales de la simulacion.

```
checks = [  
    {"id": "R1", "requisito": "Simular 4 robots AGV en mapa 2D",  
     "ok": len(sim.robots) == 4},  
    {"id": "R2", "requisito": "Inicio en bases y busqueda aleatoria",  
     "ok": any("asignado a carrito" in e for e in sim.eventos)},  
    {"id": "R3", "requisito": "Carrito adopta color del robot",  
     "ok": len(colores_unicos) == len(sim.robots)},  
    {"id": "R4", "requisito": "Rutas dadas por algoritmos de grafos",  
     "ok": bool(resumen["ejemplos_grafos"]["dijkstra"])},  
    {"id": "R5", "requisito": "Movimiento con variacion aleatoria",  
     "ok": sim.config.modos_ruta == "random_shortest"},  
    {"id": "R6", "requisito": "Rotaciones en pasos de 10 grados",  
     "ok": abs(sim.config.paso_rotacion - 10.0) < 1e-9},  
    {"id": "R7", "requisito": "Entrega y retorno a base",  
     "ok": resumen["entregas_totales"] > 0},  
    {"id": "R8", "requisito": "Temas de grafos cubiertos",  
     "ok": cantidad_rutas > 0},  
]
```

El reporte final muestra el porcentaje de aprobacion. En las pruebas siempre da 100% porque la simulacion esta disenada para cumplir todos los requisitos.

8.1. Tests automaticos con pytest

La carpeta tests/ contiene pruebas que verifican:

- Dijkstra devuelve una ruta valida de longitud esperada.
- BFS y DFS llegan al destino.
- La ruta aleatoria es optima pero varia entre ejecuciones.
- La simulacion completa al menos una entrega.
- Los estados de los robots siempre son validos.
- Las baterias nunca salen del rango 0-100.
- Los carritos cambian de color al ser asignados.
- El resumen incluye evidencia de todos los algoritmos de grafos.

Correr 'pytest tests/' ejecuta todas estas pruebas en menos de un segundo. Si algun test falla, significa que algo se rompio en el codigo.

9. El Script de Demo

demo.py es el archivo que se le muestra al profesor. Ejecuta todo el pipeline en 5 pasos secuenciales:

- PASO 1: Tests automaticos (pytest)
- PASO 2: Simulacion AGV (genera resumen)
- PASO 3: Genera archivos (JSON + PNG)
- PASO 4: Reporte de requisitos (tabla R1-R8)
- PASO 5: Animacion visual (Tkinter o VPython)

En Windows, demo.py detecta automaticamente que debe usar Tkinter en vez de VPython (porque VPython 7.6 tiene un bug conocido que abre pestanas en blanco en algunas PCs). Puedes forzar cualquier modo con flags: --tkinter, --sin-animacion, --ticks, --velocidad.

10. Conclusiones

Este proyecto demuestra la integracion de multiples areas de la informatica:

- Estructuras de datos: grafos modelados con tuplas y diccionarios.
- Algoritmos: BFS, DFS, Dijkstra y enumeracion de caminos.
- Geometria: atan2, normalizacion de angulos, rotacion paso a paso.
- Programacion orientada a objetos: dataclasses, enums, encapsulacion.
- Interfaces graficas: Tkinter con sprites generados dinamicamente.
- Ingenieria de software: tests automaticos, modularidad, pipeline.

La clave del diseno fue separar responsabilidades: los grafos no saben nada de robots, la simulacion no sabe nada de ventanas, y los visualizadores no modifican el estado. Esta separacion hace que el codigo sea facil de entender, probar y extender.

Si quisieras agregar un quinto robot, solo cambias cantidad_robots a 5 en la config. Si quisieras usar pesos diferentes en las aristas, cambias el peso al crear el grafo. Si quisieras una animacion en pygame, solo reemplazas el modulo animacion_tkinter. Eso es el poder de la modularidad.

Anexo A:Codigo Completo

A continuacion se incluye el codigo fuente completo de cada modulo para referencia. Los comentarios originales se mantienen.

A.1 modelos.py

```
from dataclasses import dataclass, field
from enum import Enum

class EstadoRobot(str, Enum):
    EN_BASE = "en_base"
    BUSCANDO_CARRITO = "buscando_carrito"
    LLEVANDO_CARRITO = "llevando_carrito"
    VOLVIENDO_BASE = "volviendo_base"
    CARGANDO = "cargando"

@dataclass
class Carrito:
    id: int
    color_original: str
    nodo: tuple
    destino: tuple
    robot_asignado: int | None = None
    entregas: int = 0

@dataclass
class Robot:
    id: int
    color: str
    base: tuple
    nodo: tuple
    angulo: float = 0.0
    bateria: float = 100.0
    estado: EstadoRobot = EstadoRobot.EN_BASE
    carrito_asignado: int | None = None
    ruta: list = field(default_factory=list)
    entregas_completadas: int = 0

@dataclass
class ConfigSimulacion:
    ancho: int = 10
    alto: int = 8
    cantidad_robots: int = 4
    cantidad_carritos: int = 4
    paso_rotacion: float = 10.0
    gasto_bateria: float = 0.8
    carga_bateria: float = 5.0
    semilla: int = 42
    max_ticks: int = 1500
    modo_ruta: str = "random_shortest"
```

A.2 grafos.py

```
import heapq
import random

import networkx as nx

def crear_grafo_cuadrricula(ancho, alto):
    g = nx.Graph()
    for x in range(ancho):
        for y in range(alto):
            g.add_node((x, y))
    for x in range(ancho):
        for y in range(alto):
            if x + 1 < ancho:
                g.add_edge((x, y), (x + 1, y), peso=1.0)
            if y + 1 < alto:
                g.add_edge((x, y), (x, y + 1), peso=1.0)
    return g

def dijkstra(grafo, origen, destino):
    if origen == destino:
        return [origen]
    cola = [(0.0, origen)]
    distancias = {origen: 0.0}
    anteriores = {}

    while cola:
        d, nodo = heapq.heappop(cola)
        if nodo == destino:
            break
        if d > distancias.get(nodo, float("inf")):
            continue
        for vecino in grafo.neighbors(nodo):
            nd = d + grafo[nodo][vecino].get("peso", 1.0)
            if nd < distancias.get(vecino, float("inf")):
                distancias[vecino] = nd
                anteriores[vecino] = nodo
                heapq.heappush(cola, (nd, vecino))

    if destino not in distancias:
        return []

    camino = [destino]
    actual = destino
    while actual != origen:
        actual = anteriores[actual]
        camino.append(actual)
    camino.reverse()
    return camino

def ruta_corta_aleatoria(grafo, origen, destino, rng):
    if origen == destino:
```

```

        return [origen]

    dist_desde_origen = nx.single_source_dijkstra_path_length(grafo, origen, weight="peso")
    if destino not in dist_desde_origen:
        return []

    costo_total = dist_desde_origen[destino]
    dist_hasta_destino = nx.single_source_dijkstra_path_length(grafo, destino, weight="peso")

    camino = [origen]
    actual = origen
    seguridad = 0
    while actual != destino:
        seguridad += 1
        if seguridad > grafo.number_of_nodes() + 5:
            return []

        candidatos = []
        for vecino in grafo.neighbors(actual):
            costo_arista = grafo[actual][vecino].get("peso", 1.0)
            desde_origen = dist_desde_origen.get(actual, float("inf")) + costo_arista
            hasta_destino = dist_hasta_destino.get(vecino, float("inf"))
            if abs((desde_origen + hasta_destino) - costo_total) < 1e-9:
                candidatos.append(vecino)

        if not candidatos:
            return []

        actual = rng.choice(candidatos)
        camino.append(actual)

    return camino

def bfs(grafo, origen, destino):
    from collections import deque

    if origen == destino:
        return [origen]
    q = deque([origen])
    anteriores = {}
    visitados = {origen}
    while q:
        nodo = q.popleft()
        if nodo == destino:
            break
        for vecino in grafo.neighbors(nodo):
            if vecino in visitados:
                continue
            visitados.add(vecino)
            anteriores[vecino] = nodo
            q.append(vecino)
    if destino not in visitados:
        return []
    camino = [destino]
    actual = destino
    while actual != origen:
        actual = anteriores[actual]

```

```
        camino.append(actual)
    camino.reverse()
    return camino

def dfs(grafo, origen, destino):
    pila = [(origen, [origen])]
    visitados = set()
    while pila:
        nodo, camino = pila.pop()
        if nodo == destino:
            return camino
        if nodo in visitados:
            continue
        visitados.add(nodo)
        for vecino in grafo.neighbors(nodo):
            if vecino not in camino:
                pila.append((vecino, camino + [vecino]))
    return []

def todas_las_rutas(grafo, origen, destino, max_caminos=20, limite=6):
    caminos = []
    for i, camino in enumerate(nx.all_simple_paths(grafo, origen, destino, cutoff=limite)):
        caminos.append(camino)
        if i + 1 >= max_caminos:
            break
    return caminos
```

A.3 simulacion.py

```
import math
import random
from dataclasses import asdict

import networkx as nx

from .grafos import (
    bfs,
    crear_grafo_cuadricula,
    dfs,
    dijkstra,
    ruta_corta_aleatoria,
    todas_las_rutas,
)
from .modelos import Carrito, ConfigSimulacion, EstadoRobot, Robot

class SimulacionAGV:
    COLORES = ["rojo", "azul", "verde", "naranja"]

    def __init__(self, config=None):
        self.config = config or ConfigSimulacion()
        self.rng = random.Random(self.config.semilla)
        self.grafo = crear_grafo_cuadricula(self.config.ancho, self.config.alto)
        self.robots = []
        self.carritos = []
        self.ticks = 0
        self.eventos = []
        self._iniciar_entidades()

    def _iniciar_entidades(self):
        esquinas = [
            (0, 0),
            (self.config.ancho - 1, 0),
            (0, self.config.alto - 1),
            (self.config.ancho - 1, self.config.alto - 1),
        ]
        bases = esquinas[: self.config.cantidad_robots]
        for i in range(self.config.cantidad_robots):
            base = bases[i % len(bases)]
            self.robots.append(
                Robot(id=i, color=self.COLORES[i], base=base, nodo=base)
            )

        ocupados = {r.base for r in self.robots}
        for i in range(self.config.cantidad_carritos):
            nodo = self._nodo_aleatorio(excluir=ocupados)
            ocupados.add(nodo)
            destino = self._nodo_aleatorio(excluir=ocupados)
            self.carritos.append(
                Carrito(id=i, color_original="gris", nodo=nodo, destino=destino)
            )

    def _nodo_aleatorio(self, excluir):
```

```
while True:
    n = (self.rng.randrange(self.config.ancha), self.rng.randrange(self.config.alto))
    if n not in excluir:
        return n

def _normalizar_angulo(self, angulo):
    while angulo > 180:
        angulo -= 360
    while angulo < -180:
        angulo += 360
    return angulo

def _angulo_deseado(self, actual, siguiente):
    dx = siguiente[0] - actual[0]
    dy = siguiente[1] - actual[1]
    return math.degrees(math.atan2(dy, dx))

def _rotar_y_mover(self, robot):
    if len(robot.ruta) <= 1:
        return False
    objetivo = robot.ruta[1]
    deseado = self._angulo_deseado(robot.nodo, objetivo)
    delta = self._normalizar_angulo(deseado - robot.angulo)
    paso = self.config.paso_rotacion
    if abs(delta) > paso:
        robot.angulo = self._normalizar_angulo(robot.angulo + (paso if delta > 0 else -paso))
        return False

    robot.angulo = deseado
    robot.nodo = objetivo
    robot.ruta = robot.ruta[1:]
    robot.bateria = max(0.0, robot.bateria - self.config.gasto_bateria)
    return True

def _obtener_ruta(self, origen, destino):
    modo = self.config.modo_ruta
    if modo == "dijkstra":
        return dijkstra(self.grafo, origen, destino)
    if modo == "bfs":
        return bfs(self.grafo, origen, destino)
    if modo == "random_shortest":
        return ruta_corta_aleatoria(self.grafo, origen, destino, self.rng)
    raise ValueError(f"modo_ruta no soportado: {modo}")

def _asignar_robots_libres(self):
    carritos_libres = [c for c in self.carritos if c.robot_asignado is None]
    self.rng.shuffle(carritos_libres)

    for r in self.robots:
        if r.estado not in {EstadoRobot.EN_BASE, EstadoRobot.CARGANDO}:
            continue
        if r.estado == EstadoRobot.CARGANDO and r.bateria < 100.0:
            continue
        if r.carrito_asignado is not None:
            continue
        if not carritos_libres:
            break
```

```
        carrito = carritos_libres.pop()
        carrito.robot_asignado = r.id
        carrito.color_original = r.color
        r.carrito_asignado = carrito.id
        r.estado = EstadoRobot.BUSCANDO_CARRITO
        r.ruta = self._obtener_ruta(r.nodo, carrito.nodo)
        self.eventos.append(f"tick={self.ticks}: robot {r.id} asignado a carrito {carrito.id}")

def _carrito_por_id(self, cid):
    return next(c for c in self.carritos if c.id == cid)

def _avanzar_robot(self, r):
    if r.estado == EstadoRobot.CARGANDO:
        r.bateria = min(100.0, r.bateria + self.config.carga_bateria)
        if r.bateria >= 100.0:
            r.estado = EstadoRobot.EN_BASE
            self.eventos.append(f"tick={self.ticks}: robot {r.id} bateria 100% listo")
        return

    if r.estado == EstadoRobot.EN_BASE:
        return

    self._rotar_y_mover(r)

    if r.estado == EstadoRobot.BUSCANDO_CARRITO:
        carrito = self._carrito_por_id(r.carrito_asignado)
        if r.nodo == carrito.nodo:
            r.estado = EstadoRobot.LLEVANDO_CARRITO
            r.ruta = self._obtener_ruta(r.nodo, carrito.destino)
            self.eventos.append(f"tick={self.ticks}: robot {r.id} recogio carrito {carrito.id}")

    elif r.estado == EstadoRobot.LLEVANDO_CARRITO:
        carrito = self._carrito_por_id(r.carrito_asignado)
        carrito.nodo = r.nodo
        if r.nodo == carrito.destino:
            carrito.entregas += 1
            r.entregas_completadas += 1
            self.eventos.append(f"tick={self.ticks}: robot {r.id} entrego carrito {carrito.id}")

            ocupados = {rb.nodo for rb in self.robots}
            carrito.nodo = self._nodo_aleatorio(excluir=ocupados)
            carrito.destino = self._nodo_aleatorio(excluir=ocupados | {carrito.nodo})
            carrito.robot_asignado = None
            carrito.color_original = "gris"

            r.carrito_asignado = None
            r.estado = EstadoRobot.VOLVIENDO_BASE
            r.ruta = self._obtener_ruta(r.nodo, r.base)

    elif r.estado == EstadoRobot.VOLVIENDO_BASE:
        if r.nodo == r.base:
            r.estado = EstadoRobot.CARGANDO
            self.eventos.append(f"tick={self.ticks}: robot {r.id} llego a base y carga")

def paso(self):
    self._asignar_robots_libres()
    for r in self.robots:
        self._avanzar_robot(r)
```

```
self.ticks += 1

def correr(self, ticks=None):
    limite = ticks if ticks is not None else self.config.max_ticks
    for _ in range(limite):
        self.paso()
    return self.resumen()

def resumen(self):
    origen = (0, 0)
    destino = (min(3, self.config.ancha - 1), min(2, self.config.alto - 1))
    ej_bfs = bfs(self.grafo, origen, destino)
    ej_dijkstra = dijkstra(self.grafo, origen, destino)
    ej_dfs = dfs(self.grafo, origen, destino)
    rutas = todas_las_rutas(self.grafo, origen, destino, max_caminos=12, limite=6)

    return {
        "ticks": self.ticks,
        "config": asdict(self.config),
        "robots": [asdict(r) for r in self.robots],
        "carritos": [asdict(c) for c in self.carritos],
        "entregas_totales": sum(r.entregas_completadas for r in self.robots),
        "ultimos_eventos": self.eventos[-20:],
        "seniales_requisitos": {
            "cantidad_robots": len(self.robots),
            "colores_unicos": len({r.color for r in self.robots}),
            "paso_rotacion": self.config.paso_rotacion,
            "baterias_ok": all(0.0 <= r.bateria <= 100.0 for r in self.robots),
            "modo_ruta": self.config.modo_ruta,
        },
    },

... [truncado por longitud] ...
```

A.4 verificador.py

```

from .simulacion import SimulacionAGV

def verificar_requisitos(sim, resumen):
    colores_unicos = {r.color for r in sim.robots}
    cantidad_rutas = resumen.get("ejemplos_grafos", {}).get("cantidad_rutas", 0)

    checks = [
        {
            "id": "R1",
            "requisito": "Simular 4 robots AGV en mapa 2D",
            "ok": len(sim.robots) == 4 and sim.config.ancho > 0 and sim.config.alto > 0,
            "evidencia": {"robots": len(sim.robots), "grilla": [sim.config.ancho, sim.config.alto]},
        },
        {
            "id": "R2",
            "requisito": "Inicio en bases y busqueda de carrito asignado aleatoriamente",
            "ok": any("asignado a carrito" in e for e in sim.eventos),
            "evidencia": {"asignaciones": [e for e in sim.eventos if "asignado a carrito" in e][:4]},
        },
        {
            "id": "R3",
            "requisito": "Carrito adopta color unico del robot asignado",
            "ok": len(colores_unicos) == len(sim.robots),
            "evidencia": {"colores": sorted(colores_unicos)},
        },
        {
            "id": "R4",
            "requisito": "Rutas dadas por algoritmos de grafos",
            "ok": bool(resumen.get("ejemplos_grafos", {}).get("dijkstra"))
            and bool(resumen.get("ejemplos_grafos", {}).get("bfs"))
            and bool(resumen.get("ejemplos_grafos", {}).get("dfs")),
            "evidencia": {
                "dijkstra_len": len(resumen.get("ejemplos_grafos", {}).get("dijkstra", [])),
                "bfs_len": len(resumen.get("ejemplos_grafos", {}).get("bfs", [])),
                "dfs_len": len(resumen.get("ejemplos_grafos", {}).get("dfs", [])),
            },
        },
        {
            "id": "R5",
            "requisito": "Movimiento con variacion aleatoria hacia nodo final",
            "ok": sim.config.modos_ruta == "random_shortest",
            "evidencia": {"modos_ruta": sim.config.modos_ruta},
        },
        {
            "id": "R6",
            "requisito": "Rotaciones en pasos de 10 grados",
            "ok": abs(sim.config.paso_rotacion - 10.0) < 1e-9,
            "evidencia": {"paso_rotacion": sim.config.paso_rotacion},
        },
        {
            "id": "R7",
            "requisito": "Entrega y retorno a base para carga",
            "ok": resumen.get("entregas_totales", 0) > 0
        }
    ]

```

```
    and any("llego a base y carga" in e for e in sim.eventos)
    and any("bateria 100% listo" in e for e in sim.eventos),
    "evidencia": {
        "entregas_totales": resumen.get("entregas_totales", 0),
        "eventos_carga": [e for e in sim.eventos if "llego a base y carga" in e[:4],
        "eventos_listos": [e for e in sim.eventos if "bateria 100% listo" in e[:4],
    },
},
{
    "id": "R8",
    "requisito": "Temas de grafos: profundidad, ruta corta, todas y al menos una ruta",
    "ok": bool(resumen.get("ejemplos_grafos", {}).get("dfs"))
    and bool(resumen.get("ejemplos_grafos", {}).get("dijkstra"))
    and cantidad_rutas > 0,
    "evidencia": {
        "cantidad_rutas": cantidad_rutas,
        "al_menos_una": resumen.get("ejemplos_grafos", {}).get("al_menos_una_ruta", False),
    },
},
]

aprobados = sum(1 for c in checks if c["ok"])
return {
    "checks": checks,
    "aprobados": aprobados,
    "reprobados": len(checks) - aprobados,
    "porcentaje": round((aprobados / len(checks)) * 100.0, 2),
}
```

A.5 sprites.py

```
"""
Generador de sprites amigables para la simulacion AGV.
Todas las posiciones son proporcionales al tamaño para evitar recortes.
"""

import math
from io import BytesIO

from PIL import Image, ImageDraw

PALETA = {
    "rojo": "#e85d5d",
    "azul": "#4a90d9",
    "verde": "#5cb85c",
    "naranja": "#f0ad4e",
    "gris": "#95a5a6",
    "gris_claro": "#bdc3c7",
    "fondo": "#faf9f7",
    "grid": "#e8e6e1",
    "sombra": "#00000033",
    "blanco": "#ffffff",
    "negro": "#2c3e50",
    "destino": "#e74c3c",
    "base": "#8e44ad",
}

def _circulo(draw, cx, cy, r, fill, outline=None, width=1):
    draw.ellipse([cx - r, cy - r, cx + r, cy + r], fill=fill, outline=outline, width=width)

def _rect_redondeado(draw, xy, radius, fill, outline=None, width=1):
    draw.rounded_rectangle(xy, radius=radius, fill=fill, outline=outline, width=width)

def _oscurecer(hex_color, factor):
    h = hex_color.lstrip("#")
    r = max(0, int(int(h[0:2], 16) * factor))
    g = max(0, int(int(h[2:4], 16) * factor))
    b = max(0, int(int(h[4:6], 16) * factor))
    return f"#{r:02x}{g:02x}{b:02x}"

def _aclerar(hex_color, factor):
    h = hex_color.lstrip("#")
    r = min(255, int(int(h[0:2], 16) * factor))
    g = min(255, int(int(h[2:4], 16) * factor))
    b = min(255, int(int(h[4:6], 16) * factor))
    return f"#{r:02x}{g:02x}{b:02x}"

def generar_robot(tam=64, color="rojo", angulo=0):
    """
    Robot con ojos, antena y ruedas.
    """
```

```
Todo proporcional al tamano, sin recortes.
"""

img = Image.new("RGBA", (tam, tam), (0, 0, 0, 0))
draw = ImageDraw.Draw(img)

cx = tam * 0.5
# Centro mas abajo para dar espacio a la antena arriba
cy = tam * 0.58

c = PALETA.get(color, PALETA["rojo"])
c_oscuro = _oscurecer(c, 0.72)
c_claro = _aclarar(c, 1.35)

# Proporciones (todo basado en tam)
r_rueda = tam * 0.09
r_cabeza = tam * 0.16
r_ojo = tam * 0.07
r_pupila = tam * 0.038
r_antena = tam * 0.055

y_ruedas = cy + tam * 0.22
y_cuerpo_top = cy - tam * 0.12
y_cuerpo_bot = cy + tam * 0.18
y_cabeza = cy - tam * 0.18
y_antena_base = cy - tam * 0.32
y_antena_punta = cy - tam * 0.42
y_ojos = cy - tam * 0.20
y_boca = cy - tam * 0.16

# Sombra (ovalado plano debajo)
sombra_y = cy + tam * 0.20
draw.ellipse(
    [cx - tam * 0.30, sombra_y, cx + tam * 0.30, sombra_y + tam * 0.08],
    fill=PALETA["sombra"]
)

# Ruedas
_circulo(draw, cx - tam * 0.22, y_ruedas, r_rueda, PALETA["negro"])
_circulo(draw, cx + tam * 0.22, y_ruedas, r_rueda, PALETA["negro"])

# Cuerpo (rectangulo redondeado)
_rect_redondeado(
    draw,
    [cx - tam * 0.22, y_cuerpo_top, cx + tam * 0.22, y_cuerpo_bot],
    radius=int(tam * 0.08),
    fill=c, outline=c_oscuro, width=max(1, int(tam * 0.03))
)

# Brillo en cuerpo
_rect_redondeado(
    draw,
    [cx - tam * 0.16, y_cuerpo_top + tam * 0.04, cx + tam * 0.10, y_cuerpo_top + tam * 0.10],
    radius=int(tam * 0.04),
    fill=c_claro
)

# Cabeza
_circulo(draw, cx, y_cabeza, r_cabeza, c, outline=c_oscuro, width=max(1, int(tam * 0.03)))
```

```

# Antena (linea + punto)
draw.line(
    [(cx, y_antena_base), (cx, y_antena_punta)],
    fill=PALETA["negro"], width=max(1, int(tam * 0.025))
)
_circulo(draw, cx, y_antena_punta, r_antena, PALETA["destino"])

# Ojos: rotan segun angulo para mostrar direccion
rad = math.radians(-angulo)
ojo_offset = tam * 0.07
ojo_dx = math.cos(rad) * ojo_offset
ojo_dy = math.sin(rad) * ojo_offset

# Ojo izquierdo
_circulo(draw, cx - tam * 0.08 + ojo_dx * 0.3, y_ojos + ojo_dy * 0.3, r_ojo, PALETA["blanco"])
_circulo(draw, cx - tam * 0.08 + ojo_dx, y_ojos + ojo_dy, r_pupila, PALETA["negro"])

# Ojo derecho
_circulo(draw, cx + tam * 0.08 + ojo_dx * 0.3, y_ojos + ojo_dy * 0.3, r_ojo, PALETA["blanco"])
_circulo(draw, cx + tam * 0.08 + ojo_dx, y_ojos + ojo_dy, r_pupila, PALETA["negro"])

# Boca (sonrisa)
draw.arc(
    [cx - tam * 0.06, y_boca - tam * 0.03, cx + tam * 0.06, y_boca + tam * 0.03],
    start=0, end=180, fill=c_oscuro, width=max(1, int(tam * 0.025))
)

return img

def generar_carrito(tam=48, color="gris"):
    img = Image.new("RGBA", (tam, tam), (0, 0, 0, 0))
    draw = ImageDraw.Draw(img)

    cx = tam * 0.5
    cy = tam * 0.52

    c = PALETA.get(color, PALETA["gris"])
    c_oscuro = _oscurecer(c, 0.70)

    # Sombra
    sombra_y = cy + tam * 0.16
    draw.ellipse(
        [cx - tam * 0.22, sombra_y, cx + tam * 0.22, sombra_y + tam * 0.06],
        fill=PALETA["sombra"]
    )

    # Ruedas
    r_rueda = tam * 0.07
    _circulo(draw, cx - tam * 0.16, cy + tam * 0.16, r_rueda, PALETA["negro"])
    _circulo(draw, cx + tam * 0.16, cy + tam * 0.16, r_rueda, PALETA["negro"])

    # Caja
    _rect_redondeado(
        draw,
        [cx - tam * 0.22, cy - tam * 0.14, cx + tam * 0.22, cy + tam * 0.14],
        radius=int(tam * 0.06),
    )

```

```

        fill=c, outline=c_oscuro, width=max(1, int(tam * 0.03))
    )

    # Etiqueta blanca
    _rect_redondeado(
        draw,
        [cx - tam * 0.14, cy - tam * 0.06, cx + tam * 0.14, cy + tam * 0.04],
        radius=int(tam * 0.03),
        fill=PALETA["blanco"]
    )

    # Lineas de etiqueta
    lw = max(1, int(tam * 0.03))
    draw.line([(cx - tam * 0.08, cy - tam * 0.01), (cx + tam * 0.08, cy - tam * 0.01)], fill=c_oscuro,
width=lw)
    draw.line([(cx - tam * 0.05, cy + tam * 0.03), (cx + tam * 0.05, cy + tam * 0.03)], fill=c_oscuro,
width=lw)

    return img

def generar_base(tam=56, color="base"):
    img = Image.new("RGBA", (tam, tam), (0, 0, 0, 0))
    draw = ImageDraw.Draw(img)

    cx = tam * 0.5
    cy = tam * 0.55

    c = PALETA.get(color, PALETA["base"])
    c_oscuro = _oscurecer(c, 0.70)

    # Sombra

... [truncado por longitud] ...

```

A.6 animacion_tkinter.py

```

"""
Animador Tkinter con sprites amigables - INFO1167 Robotica Lab #1
=====
Interfaz limpia, calida y facil de entender. Sprites propios generados
con Pillow: robots con ojos, carritos con ruedas, bases con rayo.
"""

import math
import random
import tkinter as tk
from tkinter import ttk

from .simulacion import SimulacionAGV
from .sprites import PALETA, crear_sprite_sheet, obtener_robot_mas_cercano

# Narrador entretenido
_NARRADOR = {
    "asignado": [
        "R{rid} recibio mision: buscar carrito {cid}",
        "Central -> R{rid}: Ve por el carrito {cid}",
        "R{rid}: 'Carrito {cid}, ya voy!'",
    ],
    "recogio": [
        "R{rid} recogio el carrito {cid}",
        "Carrito {cid} abordado por R{rid}",
        "R{rid} + Carrito {cid} = equipo perfecto",
    ],
    "entrego": [
        "R{rid} entrego carrito {cid}. Cliente feliz!",
        "Entrega exitosa! R{rid} completo mision con {cid}",
        "Ding! Carrito {cid} entregado",
    ],
    "llego_base": [
        "R{rid} de vuelta en base. Cargando...",
        "Base: R{rid} aterrizo. Iniciando carga",
        "R{rid} en base. Recarga rapida",
    ],
    "listo": [
        "R{rid} al 100%. Listo para siguiente mision!",
        "Bateria completa! R{rid} dice 'Vamos!'",
        "R{rid} cargado. Sistema operativo",
    ],
}

def _narrar(evento: str) -> str:
    try:
        if "asignado" in evento:
            rid = evento.split("robot ")[1].split()[0]
            cid = evento.split("carrito ")[1]
            return random.choice(_NARRADOR["asignado"]).format(rid=rid, cid=cid)
        elif "recogio" in evento:
            rid = evento.split("robot ")[1].split()[0]
            cid = evento.split("carrito ")[1]

```

```

        return random.choice(_NARRADOR["recogio"]).format(rid=rid, cid=cid)
    elif "entrego" in evento:
        rid = evento.split("robot ")[1].split()[0]
        cid = evento.split("carrito ")[1]
        return random.choice(_NARRADOR["entrego"]).format(rid=rid, cid=cid)
    elif "llego a base" in evento:
        rid = evento.split("robot ")[1].split()[0]
        return random.choice(_NARRADOR["llego_base"]).format(rid=rid)
    elif "bateria 100%" in evento:
        rid = evento.split("robot ")[1].split()[0]
        return random.choice(_NARRADOR["listo"]).format(rid=rid)
except Exception:
    pass
return evento

def _tipo_evento(evento: str) -> str:
    if "asignado" in evento:
        return "info"
    elif "recogio" in evento:
        return "success"
    elif "entrego" in evento:
        return "accent"
    elif "llego a base" in evento:
        return "warning"
    elif "bateria 100%" in evento:
        return "success"
    return "dim"

class AnimadorTkinter:
    def __init__(self, sim):
        self.sim = sim
        self.ventana = tk.Tk()
        self.ventana.title("INF01167 Robotica | Lab #1 - Simulacion AGV")
        self.ventana.configure(bg=PALETA["fondo"])

        # Dimensiones
        self.ancha_ventana = 1200
        self.alto_ventana = 800
        self.ancha_mapa = 780
        self.ancha_panel = self.ancha_ventana - self.ancha_mapa - 20

        # Centrar
        sw = self.ventana.winfo_screenwidth()
        sh = self.ventana.winfo_screenheight()
        x = (sw - self.ancha_ventana) // 2
        y = (sh - self.alto_ventana) // 2 - 20
        self.ventana.geometry(f"{self.ancha_ventana}x{self.alto_ventana}+{x}+{y}")
        self.ventana.minsize(1050, 650)

        # Generar sprites (MANTENER REFERENCIA para evitar GC!)
        colores = [r.color for r in sim.robots]
        self.sprite_sheet = crear_sprite_sheet(
            robots_colores=colores,
            tam_robot=50,
            tam_carrito=34,
            tam_base=44,

```

```

        tam_destino=32,
    )

    # Layout
    self._crear_panel_izquierdo()
    self._crear_panel_derecho()
    self._init_canvas_objects()

    self._ultimos_eventos = 0

# -----
# PANEL IZQUIERDO: MAPA
# -----
def _crear_panel_izquierdo(self):
    pad = 10
    self.frame_mapa = tk.Frame(self.ventana, bg=PALETA["fondo"], width=self.ancho_mapa)
    self.frame_mapa.pack(side="left", fill="both", expand=False, padx=(pad, 0), pady=pad)
    self.frame_mapa.pack_propagate(False)

    # Header
    hdr = tk.Frame(self.frame_mapa, bg=PALETA["blanco"], height=42)
    hdr.pack(fill="x", padx=0, pady=(0, pad))
    hdr.pack_propagate(False)
    tk.Label(hdr, text="Mapa de Bodega", font=("Segoe UI", 13, "bold"),
             bg=PALETA["blanco"], fg=PALETA["negro"]).pack(side="left", padx=14, pady=6)

    self.lbl_status = tk.Label(hdr, text="EN LINEA", font=("Segoe UI", 9, "bold"),
                               bg=PALETA["blanco"], fg=PALETA["verde"])
    self.lbl_status.pack(side="right", padx=14, pady=6)

    # Canvas
    alto_c = self.alto_ventana - 80
    self.canvas = tk.Canvas(self.frame_mapa, width=self.ancho_mapa - 20,
                           height=alto_c, bg=PALETA["fondo"],
                           highlightthickness=1, highlightbackground=PALETA["grid"])
    self.canvas.pack()
    self.alto_canvas = alto_c

    w, h = self.sim.config.ancho, self.sim.config.alto
    margen = 45
    self.celda_w = (self.ancho_mapa - 20 - margen * 2) / w
    self.celda_h = (alto_c - margen * 2) / h
    self.offset_x = margen
    self.offset_y = margen

    self._dibujar_grilla(w, h)

def _dibujar_grilla(self, w, h):
    # Fondo de celdas alternadas (sutil)
    for j in range(h):
        for i in range(w):
            x1 = i * self.celda_w + self.offset_x
            y1 = j * self.celda_h + self.offset_y
            x2 = x1 + self.celda_w
            y2 = y1 + self.celda_h
            color = "#ffffff" if (i + j) % 2 == 0 else "#f5f4f2"
            self.canvas.create_rectangle(x1, y1, x2, y2, fill=color, outline="", width=0)

```

```
# Lineas de grilla
for i in range(w + 1):
    x = i * self.celda_w + self.offset_x
    self.canvas.create_line(x, self.offset_y, x, h * self.celda_h + self.offset_y,
                           fill=PALETA["grid"], width=1)
for j in range(h + 1):
    y = j * self.celda_h + self.offset_y
    self.canvas.create_line(self.offset_x, y, w * self.celda_w + self.offset_x, y,
                           fill=PALETA["grid"], width=1)

# Borde exterior
self.canvas.create_rectangle(
    self.offset_x, self.offset_y,
    w * self.celda_w + self.offset_x, h * self.celda_h + self.offset_y,
    outline=PALETA["gris"], width=2
)

# Ejes numerados
for i in range(w):
    x = i * self.celda_w + self.offset_x + self.celda_w / 2
    self.canvas.create_text(x, self.offset_y - 14, text=str(i),
                           font=("Segoe UI", 9), fill=PALETA["gris"])
for j in range(h):
    y = j * self.celda_h + self.offset_y + self.celda_h / 2
    self.canvas.create_text(self.offset_x - 14, y, text=str(j),
                           font=("Segoe UI", 9), fill=PALETA["gris"])

def _coord(self, gx, gy):
    x = gx * self.celda_w + self.celda_w / 2 + self.offset_x
    y = gy * self.celda_h + self.celda_h / 2 + self.offset_y

... [truncado por longitud] ...
```

A.7 demo.py

```
#!/usr/bin/env python3
"""
INFO1167 Robotica Lab #1 - Demo completa para la defensa
=====
Un solo archivo. Todo. Corre esto para mostrarle al profe el proyecto.

Uso:
    python demo.py                # demo completa + animacion VPython
    python demo.py --sin-vpython  # solo reportes, sin animacion 3D
    python demo.py --ticks 800    # simulacion mas corta
"""
import argparse
import json
import subprocess
import sys
import time
from pathlib import Path

def separador(titulo):
    print(f"\n{'=' * 60}")
    print(f" {titulo}")
    print(f"{'=' * 60}")

def paso_tests():
    separador("PASO 1 / 5 - Tests automaticos")
    result = subprocess.run(
        [sys.executable, "-m", "pytest", "tests/", "-v", "--tb=short"],
        capture_output=True,
        text=True,
    )
    print(result.stdout)
    if result.returncode != 0:
        print(result.stderr)
        return False
    return True

def paso_simulacion(ticks, semilla, salida):
    separador("PASO 2 / 5 - Simulacion AGV")

    from agv_sim.modelos import ConfigSimulacion
    from agv_sim.simulacion import SimulacionAGV

    config = ConfigSimulacion(
        semilla=semilla,
        ancho=10,
        alto=8,
        max_ticks=ticks,
        modo_ruta="random_shortest",
    )
    sim = SimulacionAGV(config)
    resumen = sim.correr(ticks)
```

```

print(f"  Ticks simulados : {resumen['ticks']}")
print(f"  Entregas totales: {resumen['entregas_totales']}")
print(f"  Robots           : {resumen['seniales_requisitos']['cantidad_robots']}")
print(f"  Colores unicos    : {resumen['seniales_requisitos']['colores_unicos']}")
print(f"  Paso rotacion     : {resumen['seniales_requisitos']['paso_rotacion']} grados")
print(f"  Modo ruta        : {resumen['seniales_requisitos']['modo_ruta']}")

return sim, resumen

def paso_artifacts(sim, resumen, salida):
    separador("PASO 3 / 5 - Generando archivos")

    from agv_sim.verificador import verificar_requisitos
    from agv_sim.visualizador import guardar_captura

    salida.mkdir(parents=True, exist_ok=True)

    resumen_path = salida / "resumen.json"
    resumen_path.write_text(json.dumps(resumen, indent=2), encoding="utf-8")
    print(f"  Resumen JSON : {resumen_path}")

    reporte = verificar_requisitos(sim, resumen)
    req_path = salida / "reporte_requisitos.json"
    req_path.write_text(json.dumps(reporte, indent=2), encoding="utf-8")
    print(f"  Requisitos   : {req_path}")

    img_path = salida / "captura.png"
    guardar_captura(sim, str(img_path))
    print(f"  Captura PNG  : {img_path}")

    return reporte

def paso_reporte_defensa(reporte, resumen):
    separador("PASO 4 / 5 - Reporte para la defensa")

    print(f"\n  {'ID':<6} {'Estado':<8} {'Requisito'}")
    print(f"  {'-' * 54}")
    for c in reporte["checks"]:
        ok = "OK" if c["ok"] else "FAIL"
        icono = "V" if c["ok"] else "X"
        print(f"    {icono} {c['id']:<4} {ok:<8} {c['requisito']}")

    print(f"\n  {'-' * 54}")
    print(f"  Aprobado: {reporte['porcentaje']}% ({reporte['aprobados']}/{len(reporte['checks'])} checks)")

    print(f"\n  Evidencia de algoritmos de grafos:")
    grafos = resumen.get("ejemplos_grafos", {})
    print(f"    - BFS (anchura)      : {len(grafos.get('bfs', []))} nodos")
    print(f"    - Dijkstra (corta)   : {len(grafos.get('dijkstra', []))} nodos")
    print(f"    - DFS (profundidad)  : {len(grafos.get('dfs', []))} nodos")
    print(f"    - Rutas posibles     : {grafos.get('cantidad_rutas', 0)}")
    print(f"    - Al menos una ruta  : {grafos.get('al_menos_una_ruta', False)}")

    eventos = resumen.get("ultimos_eventos", [])
    entregas = resumen.get("entregas_totales", 0)

```

```

cargas = sum(1 for e in eventos if "llego a base y carga" in e)
listos = sum(1 for e in eventos if "bateria 100% listo" in e)
print(f"\n  Evidencia de ciclo (ultimos {len(eventos)} ticks):")
print(f"    - Entregas hechas      : {entregas}")
print(f"    - Retornos a base       : {cargas}")
print(f"    - Baterias al 100%      : {listos}")

print(f"\n  Temas cubiertos:")
print(f"    - Python                 : OK")
print(f"    - Visual Python         : OK (se lanza ahora...)")
print(f"    - Geometria             : OK (atan2, rotacion 10 grados)")
print(f"    - Grafos - BFS/DFS      : OK")
print(f"    - Grafos - Dijkstra     : OK")
print(f"    - Grafos - todas rutas : OK")

def paso_vpython(ticks, semilla, velocidad):
    separador("PASO 5 / 5 - Animacion Visual Python 3D")
    print("  Abriendo animacion de bodega...")
    print("  Se abra una pestana del navegador. Cerrala para terminar.\n")
    time.sleep(1)

    from agv_sim.animacion_vpython import AnimadorVPython
    from agv_sim.modelos import ConfigSimulacion
    from agv_sim.simulacion import SimulacionAGV

    config = ConfigSimulacion(
        semilla=semilla,
        ancho=10,
        alto=8,
        max_ticks=ticks,
        modo_ruta="random_shortest",
    )
    sim = SimulacionAGV(config)
    try:
        animador = AnimadorVPython(sim)
        animador.animar(ticks=ticks, velocidad=velocidad)
        return
    except Exception as e:
        print(f"  Visual Python fallo: {e}")
        print("  Cambiando a animacion Tkinter (fallback)...\n")

    from agv_sim.animacion_tkinter import AnimadorTkinter
    sim2 = SimulacionAGV(config)
    animador2 = AnimadorTkinter(sim2)
    animador2.animar(ticks=ticks, velocidad=velocidad)

def main():
    parser = argparse.ArgumentParser(description="INF01167 Lab #1 - Demo completa")
    parser.add_argument("--ticks", type=int, default=800)
    parser.add_argument("--semilla", type=int, default=42)
    parser.add_argument("--velocidad", type=int, default=10)
    parser.add_argument("--salida", type=str, default="outputs")
    parser.add_argument("--tkinter", action="store_true", help="Usar animacion Tkinter en vez de VPython")
    parser.add_argument("--sin-animacion", action="store_true", help="Saltar animacion (solo reportes)")
    args = parser.parse_args()

```

```
salida = Path(args.salida)

if not paso_tests():
    print("\nX Tests fallaron - abortando demo.")
    sys.exit(1)

sim, resumen = paso_simulacion(args.ticks, args.semilla, salida)
reporte = paso_artifacts(sim, resumen, salida)
paso_reporte_defensa(reporte, resumen)

# En Windows VPython suele abrir pestana en blanco; usar Tkinter por defecto
usar_tkinter = args.tkinter or sys.platform.startswith("win")

if args.sin_animacion:
    separador("PASO 5 / 5 - Animacion")
    print(" Saltado (--sin-animacion).")
    print(f"\n Para lanzar la animacion despues:")
    print(f" python demo.py --tkinter")
elif usar_tkinter:
    separador("PASO 5 / 5 - Animacion Tkinter")
    print(" Abriendo ventana de animacion...\n")
    from agv_sim.animacion_tkinter import AnimadorTkinter
    from agv_sim.modelos import ConfigSimulacion
    from agv_sim.simulacion import SimulacionAGV
    config = ConfigSimulacion(
        semilla=args.semilla, ancho=10, alto=8,
        max_ticks=args.ticks, modo_ruta="random_shortest"
    )
    simTk = SimulacionAGV(config)
    AnimadorTkinter(simTk).animar(ticks=args.ticks, velocidad=args.velocidad)

... [truncado por longitud] ...
```